WristMatch - Software Requirements Specification

Version: 1.0 | Date: 11 October 2026 | Owner: Zubair Hussain

Status: Proposed project; no implemented features are implied.

Target: YouCam API Skin AI & eCommerce VTO Hackathon

Audience: Cursor AI and the implementing developer

1. Product purpose

Build a responsive web application that helps shoppers choose watches by filtering a product catalogue, generating virtual try-on previews on their own wrist photo, comparing products and saving favourites. Support bracelets as a later extension after the watch journey is complete.

Consumer problem: product-only photographs make it difficult to judge personal appearance and compare styles before buying. WristMatch combines visual previews with transparent budget and occasion matching.

Primary journey: choose preferences > browse matching products > upload wrist photo with consent > generate watch preview > compare up to three results > favourite or visit retailer.

Success means a new visitor completes this journey without developer assistance. Visual previews are AI generated and are not measurements, guarantees of physical fit, or proof of product authenticity. Conversion and return-rate improvements are hypotheses to evaluate, not existing achievements.

2. Scope and priorities

Priority	Included capabilities
P0 / MVP	Watch catalogue, filters, upload and consent, real Watch API integration, job status, result display, compare, favourites, retailer links, secure admin product CRUD, deletion and expiry, responsive UI
P1	Bracelet adapter and catalogue, explicit opt-in sharing, camera capture, basic aggregate usage dashboard
Future	Shopify catalogue sync, multilingual interface, natural-language shopping assistant, merchant widget

Exclude from MVP: payments, checkout, inventory reservation, live AR tracking, custom AI model training, exact wrist sizing, combined jewellery rendering, skin analysis, automatic biometric or demographic inference, multi-tenant billing, social login and mandatory shopper registration.

Use an image-based try-on experience. Do not label it live AR or real-time tracking.

3. Roles

- Guest shopper: browse, submit their own permitted photo, generate previews, compare, favourite and delete their data within their session.

- Administrator: authenticate and manage catalogue records and product images; view aggregate job health. Admin access does not grant access to shopper photos.

- Background worker: process queued jobs, call provider, update state and remove expired private assets.

4. Technology and structure

Frontend: React with TypeScript, Vite, Tailwind CSS, React Router and TanStack Query. Use accessible native controls where possible.

Backend: ASP.NET Core Web API, C#, EF Core and SQL Server. Select compatible supported stable versions available in the developer environment, pin versions and document prerequisites; do not assume a specific SDK is installed.

Admin authentication: ASP.NET Core Identity with an HttpOnly secure cookie. Protect authenticated mutations with antiforgery tokens. Use a narrow CORS allowlist if frontend and API have separate origins; same-origin deployment is preferred.

Storage: private server directory outside the static web root for shopper images/results; public approved catalogue images may live in static storage. Abstract storage to permit later object-storage deployment.

Background processing: database-backed jobs processed by a hosted worker, with an atomic lease. Never hold an HTTP request open while waiting for YouCam completion.

Tests: xUnit for meaningful backend behaviour and Playwright for the core browser journey. Provider tests use a stub; live testing is manual and opt-in to control credits.

Suggested repository:

```
/frontend/src/{pages,components,features,services,types}
/backend/WristMatch.Api
/backend/WristMatch.Application
/backend/WristMatch.Infrastructure
/backend/WristMatch.Domain
/backend/WristMatch.Tests
/tests/e2e
/docs/provider-integration.md
/docs/deployment.md
README.md
.env.example
```

Keep separation pragmatic. Do not create microservices or unnecessary generic repositories.

5. Screens and navigation

Route	Required content
/	Value proposition, “Try a watch” CTA, short explanation, featured products
/catalog	Filters, sort, product cards, prices, favourite and try-on actions
/products/:slug	Images, factual specifications, recommendation reasons, try-on and retailer CTA
/studio	Photo guide, consent, upload preview, selected product, job state, result, replace/delete photo
/compare	Up to three products with separate previews, prices, materials and recommendation reasons
/favourites	Saved catalogue products; no implied permanent image retention
/privacy	Processing purpose, YouCam transfer, retention and deletion limitations
/admin/login	Admin sign-in
/admin/products	Protected catalogue list/create/edit/archive

Visual direction: contemporary jewellery retail UI, ivory background, charcoal typography, restrained gold accents, large product imagery and clear primary actions. Ensure contrast and focus visibility. Design from 360 px mobile through desktop; comparison may scroll horizontally with labelled cards. Never invent branded product imagery or show a fabricated AI result as a genuine integration.

6. Functional requirements and acceptance criteria

FR-01 Catalogue

Display 10-15 approved sample watches. Each product has name, category, price/currency, colour, material, occasion tags, product image, optional case diameter and retailer URL. Clearly label fictional catalogue products and sample prices. Use owned/licensed images and retain attribution information.

Acceptance: only active products appear publicly; archived products cannot start new jobs; missing optional specifications display “Not provided”.

FR-02 Preferences and filters

Support maximum budget, occasion (Office, Wedding, Casual, Gift), colour and material. MVP uses one currency, PKR; schema records currency and prohibits comparison across currencies without conversion support. Sort by recommended, price ascending or descending.

Acceptance: filters combine consistently; no product exceeds the selected maximum budget; empty results explain how to relax filters; reset restores catalogue.

FR-03 Explainable recommendations

Use deterministic catalogue rules, not an LLM. Budget is a hard filter. Rank surviving products by occasion match (3 points), colour match (2), material match (1); ties resolve by price ascending then stable product ID. Each score reason comes from stored metadata. Filters explicitly selected by the shopper remain hard constraints; scoring can rank the remaining products by preferences.

Acceptance: identical inputs yield identical order; explanation does not claim suitability from inferred age, gender or skin tone. No unverified health, quality or sizing claims.

FR-04 Guest session

Create a cryptographically random opaque session token in an HttpOnly cookie; store its hash server-side. Session lifetime: 24 hours, configurable. All private media and jobs require matching ownership. Session loss means private data is not recoverable through an account.

Acceptance: a second browser/session cannot retrieve, delete or inspect the first session's jobs or photos, even with known UUIDs.

FR-05 Photo upload and consent

Require an unchecked consent checkbox before upload: “I have permission to use this photo and agree to send it to YouCam to generate my preview.” Link processing details and retention.

MVP accepts JPEG/PNG only, maximum 8 MiB, long side no more than 4096 pixels, plus a configurable decoded pixel-count limit. Validate actual decoded content, not extension alone. Normalize orientation, remove EXIF and re-encode. Reject decompression bombs and corrupt files. Generate storage names; ignore client filenames.

Show provider-aligned pose instructions: back of wrist visible, all five fingers shown, clear lighting and no occlusion. Do not promise automatic pose detection unless separately implemented; provider may reject a photo.

Acceptance: missing consent or invalid content is rejected on both client and server; replacing a photo does not cause automatic chargeable generation.

FR-06 Start try-on

Shopper explicitly clicks Generate Preview after choosing a photo and active watch. Backend validates ownership, product image readiness, consent, rate limits and session quota. Return local job ID immediately with HTTP 202.

Use Idempotency-Key for submissions; enforce unique session/key and return existing job on repeated request. Reusing the key with different inputs returns 409. A new preview requires a deliberate action and new key.

Acceptance: double-click/retry of the same client request creates one local job and at most one provider submission unless ambiguous provider behaviour requires reconciliation.

FR-07 Processing and states

Use Queued > Preparing > Submitted > Processing > Succeeded; terminal alternatives Failed, Expired or Cancelled. Include timestamps, failure code and safe user message. Timeout after a configurable 180-second active processing deadline; provider state can be reconciled by stored provider task ID without a second create request.

Client polls local API every 3 seconds while visible and pauses when hidden; resume on return. Worker polling/backoff must honour provider limits and Retry-After where supplied.

Acceptance: refresh restores the selected session job; success stops polling; failure presents actionable guidance; no simulated percentage is displayed.

FR-08 Results

Display original and generated preview with product identity and price. Serve private images through ownership-checked application endpoints. Copy completed provider outputs into private application storage before provider URLs expire, subject to provider usage terms.

Acceptance: result is tied to the selected product/job; image can be zoomed; it carries the label “AI-generated visual preview - appearance and fit may vary.”

FR-09 Compare

Compare up to three catalogue products. Display product metadata even when preview is absent. Generate each chosen preview separately; show the number of requested generations before starting a batch and apply quota checks per job.

Acceptance: adding a fourth requires removal; original input photo is reused with consent; failed preview does not hide other results; do not combine separate results into an alleged provider-generated jewellery set.

FR-10 Favourites and purchase intent

Store favourite product IDs in localStorage for guests. Provide “View at retailer” rather than an internal Buy Now checkout. Validate links server-side as HTTPS; no arbitrary shopper-supplied URL retrieval. Open external links with noopener/noreferrer.

Acceptance: favourites survive reload, archived products show unavailable state, and no payment collection is implemented.

FR-11 Administration

Authenticated admin can create/edit/archive products, upload approved product images and maintain factual specs and licensing notes. Seed admin through a setup command/environment secrets, never committed passwords. Validate watch images against provider guidance manually before enabling try-on: upright front view, unobstructed face, realistic front-facing strap, no rear clasp.

Acceptance: public users receive 401/403 for admin calls; invalid image/product data does not become active; edits are recorded in an audit event without secrets or shopper media.

FR-12 Deletion and retention

Provide Delete photo/result and Delete my session data. Local photos/results expire after 24 hours by default; worker deletes expired files at least hourly. A missing/expired asset returns 404/410 without a signed URL.

Deleting a source photo cancels local unsubmitted dependent jobs. For submitted jobs, mark cancellation requested, prevent result retrieval/persistence and clean up any arriving output. Do not imply a provider task is cancellable without documentation. Best-effort provider deletion must follow confirmed supported endpoints; explain provider retention separately.

Acceptance: deletion immediately blocks local access, physical cleanup completes within one hour, and worker cannot recreate deleted outputs. Never claim instant deletion from provider infrastructure.

FR-13 Optional sharing (P1)

Private by default. Sharing requires a separate explicit action explaining public access. Create a high-entropy revocable share token, store its hash, default expiry 24 hours and display generated output only, not original photo. Never expose raw session tokens.

Acceptance: unshared results cannot be viewed anonymously; expiry/revocation blocks access. Do not implement public sharing implicitly.

7. YouCam integration contract

Authoritative starting reference, reviewed 11 October 2026:

https://docs.perfectcorp.com/reference/ai_watch

The current Watch guide documents:

- POST /s2s/v2.0/file for file preparation/upload workflow.

- POST /s2s/v2.0/task/2d-vto/watch to create a watch task.

- GET /s2s/v2.0/task/2d-vto/watch/{task_id} to poll it.

- Authorization: Bearer YOUR_API_KEY.

- Wrist source via src_file_id or src_file_url; watch reference via ref_file_ids or ref_file_urls.

- Task polling until provider task_status becomes success or error.

- JPEG/PNG images, long side <=4096 and file size <10MB; application deliberately uses a stricter 8 MiB limit.

This is a workflow summary, NOT a complete provider request schema. Before writing the adapter, inspect the current versioned reference, response schema, API base URL, file upload sequence, required fields, unit cost, limits, result URL fields and deletion support. Record verified details in docs/provider-integration.md. Do not infer JSON bodies, confuse local API routes with provider routes, or treat the file preparation endpoint as necessarily accepting raw multipart bytes.

Implement IYouCamTryOnProvider with operations to prepare images, submit a watch task and query task state. Map provider states/errors to local states. Persist provider task ID immediately after a successful create response. All calls originate server-side through a typed HttpClient; API key is never sent to frontend or logs.

Do not blindly retry task creation after a timeout: a chargeable task may already exist. Mark an ambiguous submission and reconcile if possible; otherwise require an explicit user retry with clear messaging. Retry read/status requests and safe preparation requests only according to confirmed semantics. Rate-limit errors honour Retry-After.

Provider failures such as PHOTO_DETECTION_FAIL, OBJECT_DETECTION_FAIL, PHOTO_CHECK_INVALID and INPUT_ERROR should become safe messages about wrist pose, product image or file validity. Keep raw diagnostic details restricted and redacted.

For development implement a stub adapter returning clearly labelled test fixtures. Stub mode shows a persistent “Demo mode - no live YouCam generation” badge. Production defaults to real mode and fails safely if credentials are missing. The hackathon demonstration must include actual working YouCam integration.

Bracelet API documentation starting point:

https://docs.perfectcorp.com/reference/ai_bracelet

Verify its independent contract before implementing P1; do not reuse watch payload fields without checking.

8. Application API (our own contract)

All routes prefixed /api/v1. Use application/problem+json errors with status, code, safe detail and traceId. UUID IDs do not replace authorization. Dates are UTC ISO 8601. Pagination page/pageSize, default 12, maximum 50.

Method / route	Behaviour
POST /sessions	Create/reuse guest cookie session
GET /products	Public filtered/paginated catalogue
GET /products/{id}	Public active product details
POST /photos	Multipart upload with consentVersion; creates private photo
GET /photos/{id}/image	Owned original normalized image
DELETE /photos/{id}	Owned deletion/cancellation policy
POST /tryons	JSON photoId/productId; Idempotency-Key required; 202 job response
GET /tryons/{id}	Owned state, productId, safe error and local result path when ready
GET /tryons/{id}/image	Owned generated output; no provider credentials/URLs
DELETE /tryons/{id}	Revoke access and remove local output
DELETE /sessions/current/data	Delete session media/jobs; expire cookie
POST /admin/login	Cookie sign-in with throttling
POST /admin/logout	End admin session
GET /admin/products	Protected all products
POST /admin/products	Protected create
PUT /admin/products/{id}	Protected update with concurrency guard
POST /admin/products/{id}/image	Protected approved image upload
DELETE /admin/products/{id}	Protected archive
GET /health/live	Liveness without internal details
GET /health/ready	Database/worker configuration readiness, no charged provider call

Try-on 202 example: {"id":"UUID","status":"Queued","pollUrl":"/api/v1/tryons/UUID"}.

Photo response: {"id":"UUID","expiresAt":"UTC timestamp","previewUrl":"owned local route"}.

Statuses: 400 invalid input, 401 admin not signed in, 403 forbidden, 404 inaccessible/missing private resource, 409 conflicting state/idempotency, 413 oversized upload, 415 unsupported media, 429 quota, 503 service unavailable.

9. Data model

Entity	Main fields and constraints
Product	Id UUID, Slug unique, Name <=120, Category enum Watch/Bracelet, Description <=2000, Price decimal(18,2) >=0, Currency char(3), Colour, Material, CaseDiameterMm nullable >0, ImageStorageKey, RetailerUrl nullable, IsActive, CreatedAt, UpdatedAt, RowVersion
ProductOccasion	ProductId FK, Occasion enum; composite unique key
ProductAssetLicence	ProductId FK, Source, LicenceNote, Attribution, approval timestamp
GuestSession	Id UUID, TokenHash unique, CreatedAt, ExpiresAt

Entity	Main fields and constraints
PhotoAsset	Id UUID, SessionId FK, StorageKey private, MimeType, Width, Height, ByteSize, ConsentVersion, ConsentAt, ExpiresAt, DeletedAt nullable
TryOnJob	Id UUID, SessionId FK, PhotoId FK, ProductId FK, ProductSnapshotJson, State, IdempotencyKey, RequestHash, ProviderTaskId nullable, ProviderSubmissionState, ErrorCode, SafeMessage, CreatedAt, UpdatedAt, DeadlineAt, NextPollAt, LeaseUntil, WorkerId, AttemptCount, CancellationRequestedAt, RowVersion
TryOnResult	Id UUID, JobId unique FK, StorageKey private, CreatedAt, ExpiresAt, DeletedAt nullable
AdminUser	ASP.NET Core Identity tables; hashed credentials
AuditEvent	Id, AdminUserId, Action, ProductId optional, Timestamp; no secrets/photo payload

Indexes: active product/category/price; session token hash; job state/NextPollAt; lease expiry; asset expiry. Unique session+idempotency key. Snapshot product metadata at job creation so comparison retains correct identity after edits. Restrict hard product deletion while referenced; archive instead.

10. Security, privacy and reliability requirements

- HTTPS outside local development. Secrets via user-secrets/environment or hosting secret manager. Commit examples containing placeholders only.

- Limit guest uploads to 20/hour and try-on creation to 10/hour per session, plus IP abuse limits and a configurable global daily cap; count queued/submitted jobs toward quota. Defaults are proposed application controls, not provider limits.

- Max 1 active provider submission per session and 2 global active jobs by default. Configure according to verified API limits and available units.

- Rate-limit sign-in and lock out repeated failures using Identity. Restrict admin endpoints and protect cookie mutations from CSRF.

- Do not accept arbitrary remote image URLs from guests. If provider result URLs are fetched, restrict to verified HTTPS provider hosts, block redirects to untrusted/internal hosts and enforce output size/time limits.

- Log trace IDs, durations and error codes; never photo bytes, tokens, API keys, signed URLs or full provider bodies. No analytics tied to personal photo content.

- Persist queues and recover leases after restart. Atomic lease acquisition prevents two workers submitting one job. Provider submission uncertainty must remain visible in job state rather than hidden by retries.

- Catalogue requests target <500 ms server response for the demo dataset; upload acknowledgement <3 seconds after transfer on a normal development setup. AI generation latency is provider-dependent; do not guarantee it.

- Support keyboard navigation, labelled controls, visible focus, accessible status announcements and WCAG 2.2 AA as a design target.

- Private image endpoints send Cache-Control: no-store. Downloads/favourites must not unintentionally make private images public.

11. Verification matrix

Test	Required outcome
Catalogue filter/ranking	Budget respected, consistent ordering, factual explanation
Upload checks	Fake extension, oversized, corrupt and oversized decoded image rejected
Ownership	Another session cannot access job/photo/output/delete
Idempotency	Repeated same request does not create a second provider submission

Test	Required outcome
Async worker	Queue persists through restart; leases avoid duplicate processing
Provider success/failure	Stub states map correctly; safe errors; terminal polling stops
Quota	Rejected request never calls provider
Delete while processing	Result cannot reappear after deletion/cancellation
Admin security	Unauthorized CRUD fails; invalid CSRF mutation fails
Browser journey	Filter > upload > submit > result > compare > favourite works
Missing key/demo mode	Explicit mode badge or unavailable state; no false live results
Manual live smoke test	One real watch task generates a usable preview with verified request/response

Do not run chargeable live integration calls automatically in CI.

12. Delivery stages

1. Provider feasibility: test allowed product/wrist inputs in Playground, verify API schema/units and document findings.

2. Scaffold: repo, configuration examples, migrations, catalogue seeds, frontend catalogue and admin auth.

3. Vertical slice: consent/upload, private ownership, database job, worker and real watch provider adapter, one result.

4. Decision flow: preferences, ranking reasons, compare and favourites; retailer link.

5. Reliability: idempotency, quotas, retry/ambiguity handling, expiry, cancellation and security tests.

6. Polish: mobile/accessibility checks, deployment README, live smoke test, screenshots and 1-3 minute demo.

7. P1 only after MVP acceptance: bracelets and optional sharing.

Suggested schedule: 11-13 Oct feasibility/scaffold; 14-19 Oct vertical slice; 20-23 Oct decision flow; 24-28 Oct verification/polish; 29 Oct-1 Nov demonstration/submission preparation. These are planning targets, not commitments.

13. Definition of done

- Fresh clone runs from README with documented environment, database migrations and approved sample assets.

- A real Watch API request completes the full private shopper journey.

- No API keys, passwords or user photos are committed to git.

- Core tests pass, including ownership, idempotency and deletion during processing.

- Production mode never silently substitutes stub results.

- Known limitations and preview disclaimer are visible and documented.

- Source repository, setup instructions, screenshots and a short working demonstration are prepared; submission copy accurately describes completed features.

14. Cursor AI implementation prompt

Paste the following into Cursor Agent after placing this file at docs/WristMatch_SRS_Cursor.md:

```
Act as a senior full-stack engineer. Implement WristMatch according to
@docs/WristMatch_SRS_Cursor.md using React + TypeScript + Tailwind and
ASP.NET Core + EF Core + SQL Server.

First inspect the workspace and installed SDKs. Preserve existing work.
Read the SRS and produce a short implementation checklist. Then begin the
MVP in delivery-stage order, completing and verifying each vertical slice.
```

```
Do not build P1 features before the MVP works.

Before the real YouCam adapter, verify the current official Watch API
contract and record its exact base URL, payload/upload/status schemas,
unit cost and documented limits in docs/provider-integration.md.
If browsing/API access is unavailable, implement the clearly labelled
stub behind the interface and explicitly list the provider contract blocker.
Never invent endpoints, schemas, credentials or successful live results.
Ask me to configure secrets locally; never ask me to paste an API key into
chat and never expose it in the browser or repository.

Implement ownership checks, async persistent jobs, idempotency, safe errors,
consent, expiry/deletion, quotas and admin security as part of the vertical
slice. Use deterministic recommendations; no extra LLM API is needed.
Seed only owned/licensed or clearly labelled sample assets and prices.
Write meaningful tests for the core behaviours in section 11.

Deliver runnable source, migrations, .env.example/config examples, setup and
deployment instructions, and a list of implemented versus pending features.
At each stage run the relevant checks and fix failures before proceeding.
Do not claim the real integration is validated unless an authorized live
smoke test actually succeeds. Do not deploy or submit the hackathon entry
without my instruction.
```

15. Open decisions and dependencies

- Verify account access to Watch API, actual unit costs and adequate credit balance before choosing demo volume.
- Supply permitted wrist and catalogue images; validate their quality in Playground.
- Confirm hosting environment, SQL connection and supported .NET SDK before deployment.
- Verify provider terms and retention/deletion capabilities before publishing privacy text.
- WristMatch is a working name; trademark/domain availability has not been checked.

16. References

Provider Watch API: https://docs.perfectcorp.com/reference/ai_watch

Provider Bracelet API: https://docs.perfectcorp.com/reference/ai_bracelet

Hackathon requirements: https://youcam-api-skin-ai-ecommerce.devpost.com/

Provider facts were reviewed on 11 October 2026. Recheck versioned documentation during implementation. Application quotas, retention, architecture and schedule in this SRS are proposed project requirements rather than provider promises.